

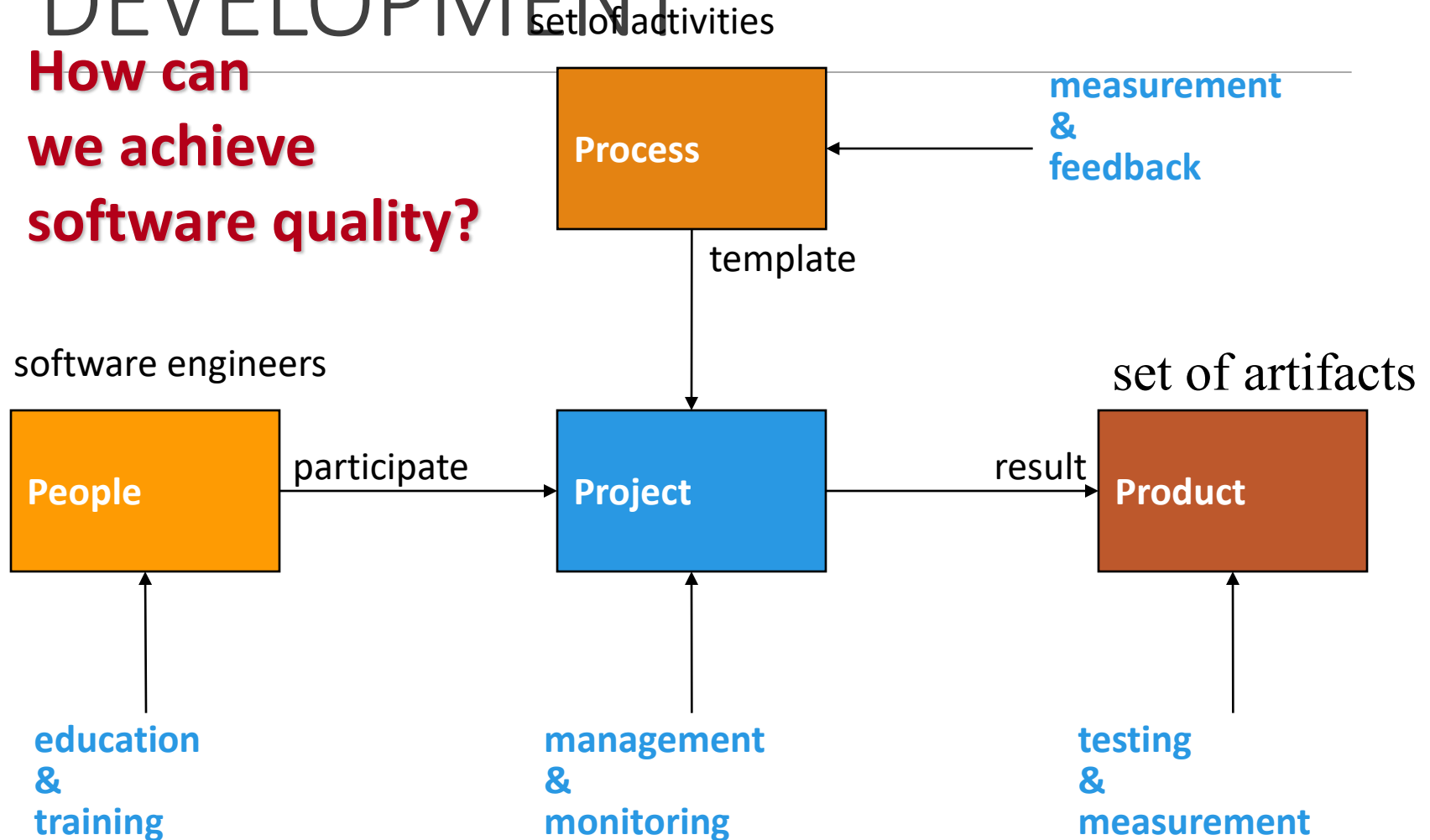
SOFTWARE ENGINEERING & PROJECT MANAGEMENT

UNIT-II

SOFTWARE QUALITY ASSURANCE

THE FOUR P's IN SOFTWARE DEVELOPMENT

**How can
we achieve
software quality?**



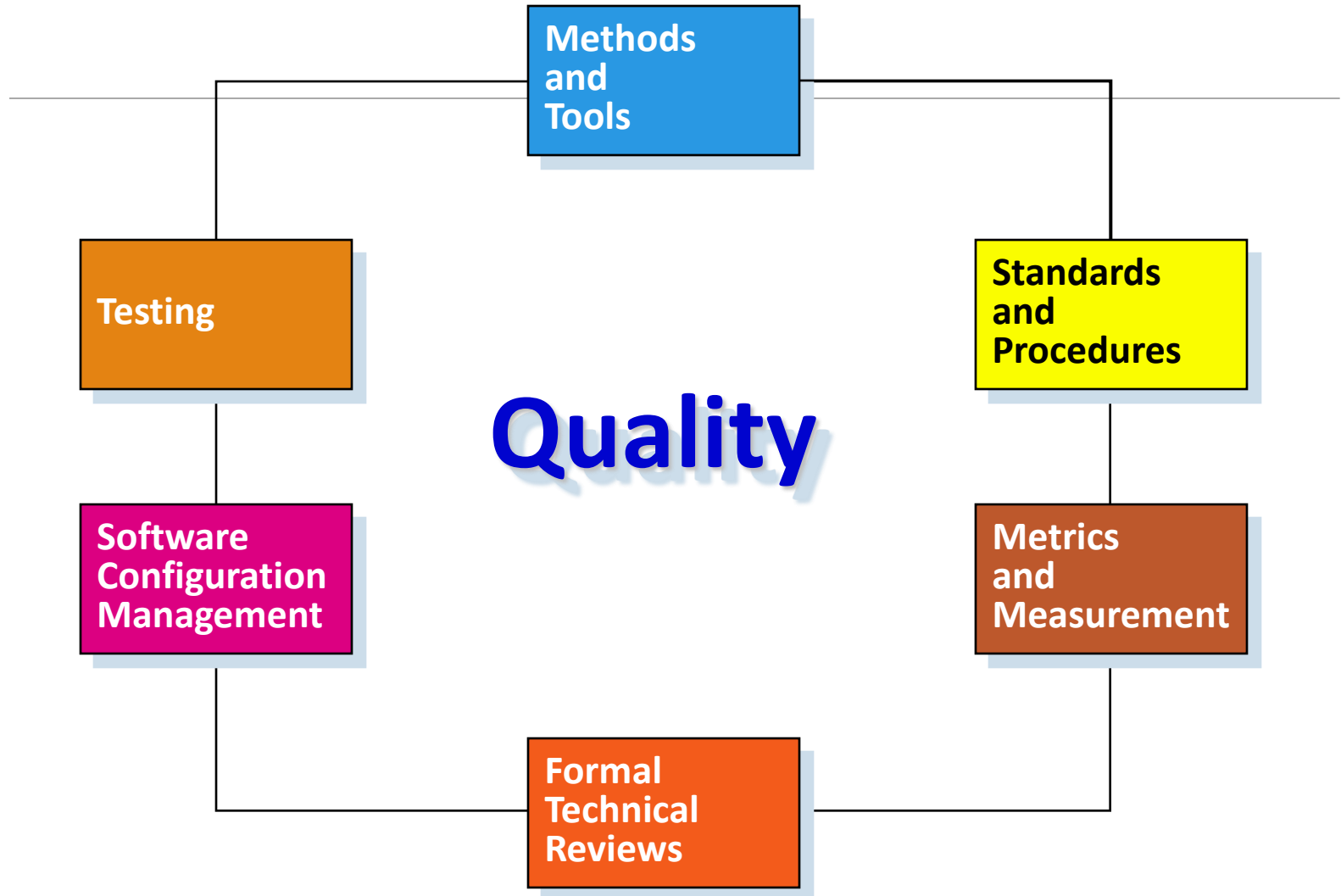
SOFTWARE QUALITY ASSURANCE (SQA)

*Quality assurance consists of those **procedures, techniques, and tools** applied by professionals to ensure that a product **meets or exceeds pre-specified standards** during its development cycle.*

E.H. Bersoff

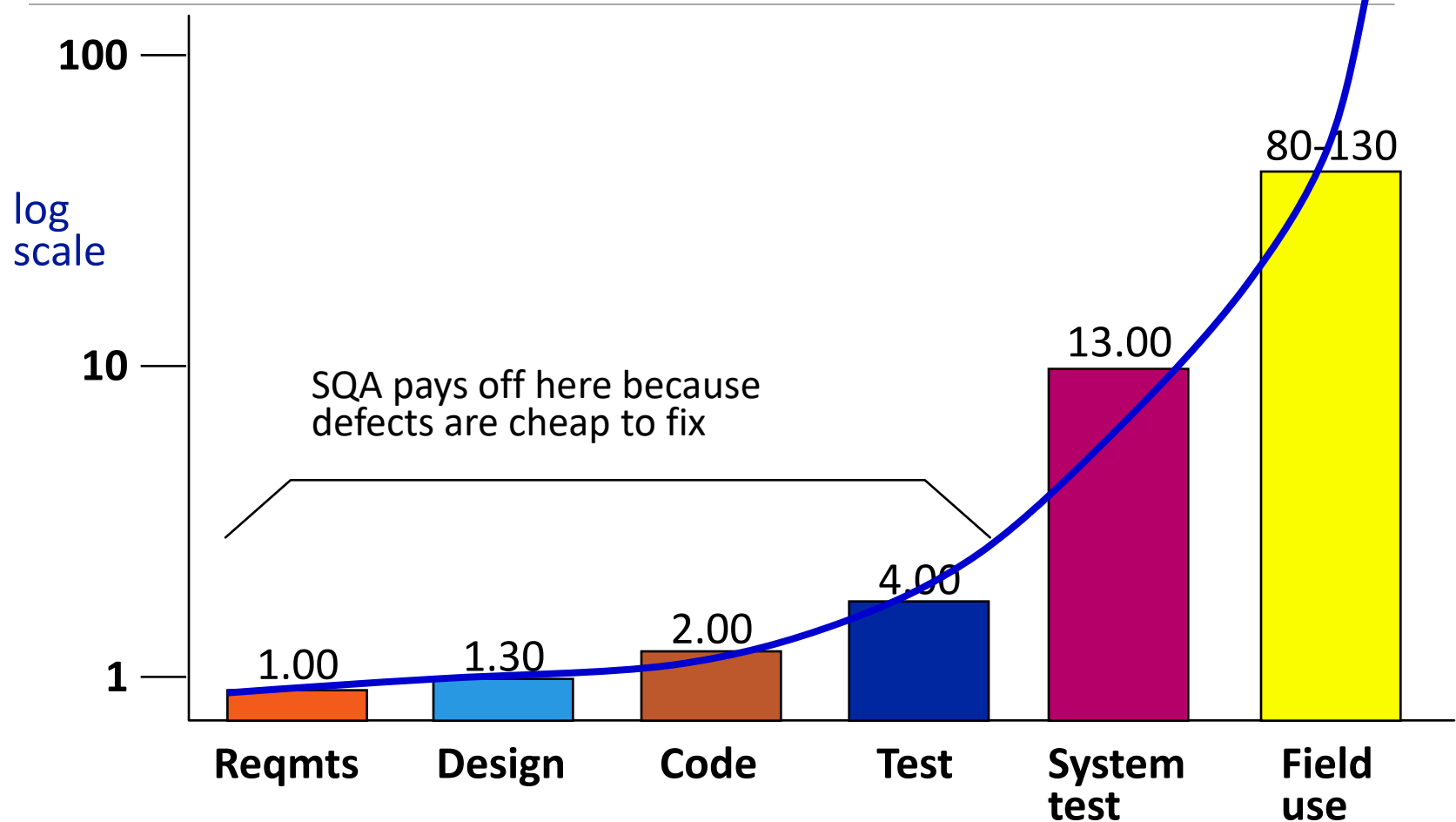
- ➔ It is an **essential activity** for any business that produces products used by others.
- ➔ It needs to be **planned** and **systematic**.
(It does not just happen)
- ➔ It needs to be **built into** the development process.
(A natural outcome of software engineering)
- ➔ **Continuous improvement** is the overall goal.

SQA — AN UMBRELLA ACTIVITY



WHY SQA ACTIVITIES PAY OFF

cost to find
and fix a defect



SOFTWARE QUALITY

What is software quality and how do we measure it?

customer's viewpoint	→	meets specifications
developer's viewpoint	→	easy to maintain, test, . . .

➔ **Software quality is not just about meeting specifications and removing defects!**

Other attributes of software that affect its quality:

- | | | |
|---------------|---------------------|----------------|
| – safety | – understandability | – portability |
| – security | – testability | – usability |
| – reliability | – adaptability | – reusability |
| – resilience | – modularity | – efficiency |
| – robustness | – complexity | – learnability |

We need to select critical quality attributes early in the development process and plan for how to achieve them.

PRINCIPLES OF SOFTWARE QUALITY ASSURANCE

1. We have a set of **standards** and **quality attributes** that a software product must meet.

➔ **There is a goal to achieve.**

2. We can **measure the quality** of a software product.

➔ **There is a way to determine how well the product conforms to the standards and the quality attributes.**

← **key point**

3. We **track** the values of the quality attributes.

➔ **It is possible to assess how well we are doing.**

4. We use information about software quality to **improve the quality** of future software products.

➔ **There is feedback into the software development process.**

SOFTWARE STANDARDS

Why are software standards important?

1. encapsulate **best** (or **most appropriate**) practices
 - acquired after much trial and error → helps avoid previous mistakes
 2. provide a **framework** around which to implement SQA process
 - ensures that best practices are properly followed
 3. assist in ensuring **continuity of project work**
 - reduces learning effort when starting new work
- **product standards**: define the characteristics all product artifacts should exhibit so as to have quality
 - **process standards**: define how the software process should be conducted to ensure quality software

**Standards
Handbook
needed!**

**Each project needs to decide which standards should be:
ignored; used as is; modified; created.**

SOFTWARE METRICS

metric: any type of measurement that relates to a software system, process or related artifact

control metrics - used to plan, manage and control the development process
(e.g., effort expended, elapsed time, disk usage, etc.)

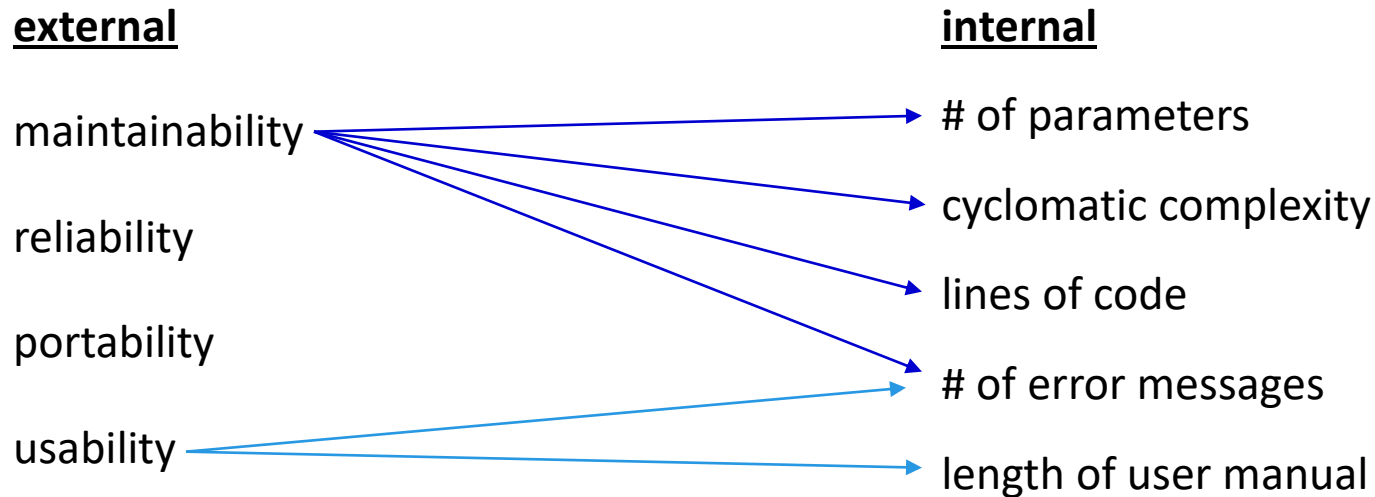
predictor metrics - used to predict an associated product quality
(e.g., cyclomatic complexity can predict ease of maintenance)

external attribute: something we can only discover after the software has been put into use (e.g., ease of maintenance)

internal attribute: something we can measure directly from the software itself (e.g., cyclomatic complexity)

SOFTWARE METRICS (cont'd)

We want to use internal attributes to predict the value of external attributes.



- Problems:**
1. **hard to formulate and validate relationships between internal and external attributes**
 2. **software metrics must be collected, calibrated, interpreted**

PRODUCT QUALITY: DESIGN QUALITY METRICS

For a design component, the key quality attribute is maintainability.

For design components maintainability is related to:

- **cohesion** - how closely related is the functionality of the component?
- **coupling** - how independent is the component?
- **understandability** - how easy is it to understand what the component does?
- **adaptability** - how easy is it to change the component?

Problem: most of these cannot be measured directly, but it is reasonable to infer that there is a relationship between these attributes and the “**complexity**” of a component

➔ **measure complexity**

How?

PRODUCT QUALITY: DESIGN QUALITY METRICS

a) Structural fan-in/fan-out

fan-in – number of calls to a component by other components

fan-out – number of components called by a component

➔ high fan-in => high coupling

➔ high fan-out => calling component has high complexity

b) Informational fan-in/fan-out

- consider also the **number of parameters** passed plus **access to shared data structures**

complexity = component-length x (fan-in x fan-out)²

➔ It has been validated using the Unix system

➔ It is a useful predictor of **effort** required for **implementation**

PRODUCT QUALITY: DESIGN QUALITY METRICS

c) IEEE Standard 982.1-1988

looks at: **subsystem properties** (number of subsystems and degree of coupling)
 database properties (number of attributes and classes)

➔ compute a **design structure quality index—DSQI** → (0-1)

➔ used to **compare with past designs**; if DSQI is too low, further design work and review may be required

- we can also consider **changes** made throughout the lifetime of the software and compute how **stable** the product is (i.e., how many changes have been made in subsystems in the current release)

➔ define a **software maturity index—SMI** → (0-1)

➔ as SMI approaches 1, the product begins to stabilize

PRODUCT QUALITY: PROGRAM QUALITY METRICS

For an implementation component, the key quality attributes are reliability and difficulty of implementation.

a) Halstead's Software Science

Looks at operators and operands in an implementation component and calculates values for component volume, V , component difficulty, D , and effort, E , required to implement the component.

n_1 = number of unique operators in a component

n_2 = number of unique operands in a component

N_1 = the total number of operators

N_2 = the total number of operands

$L = N_1 + N_2$ (component length)

$V = L * \log_2(n_1 + n_2)$ (component volume in bits)

$D = (n_1/2) * (N_2/n_2)$ (difficulty of implementing the component)

$E = V * D$ (effort required to implement the component)

PRODUCT QUALITY: PROGRAM QUALITY METRICS

b) McCabe's Complexity Metric

Looks at **control flow** in a component.

Cyclomatic Complexity → measures component's **logical** complexity

➔ an indication of the **testing difficulty** of a component

Studies have shown a **distinct relationship** between the **Cyclomatic Complexity** of a component and the **number of errors** found in the source code, as well as the **time required to find and fix errors**.

c) Other program quality metrics

- length of code
- length of identifiers
- depth of conditional nesting

Standards can be established to **avoid** complex components and/or **highlight** problem components.

PRODUCT QUALITY: FORMAL APPROACHES

a) Proving programs/specifications correct

- logically prove that requirements have been correctly transformed into programs
(e.g., prove assertions about programs)

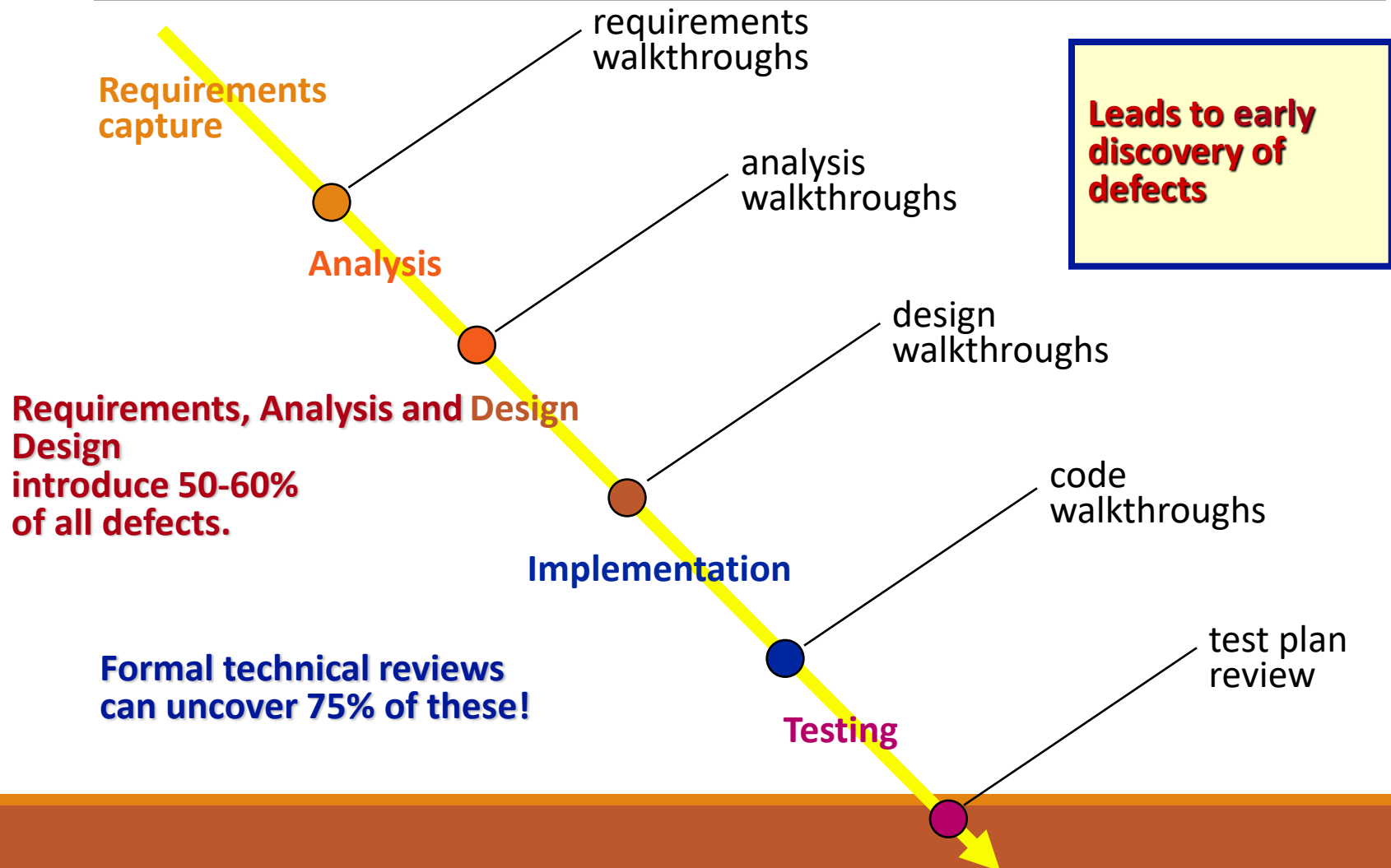
b) Statistical Quality Assurance

- categorize and determine cause of software defects
- **80-20 rule** → 80% of defects can be traced to 20% of causes
- isolate and correct 20% of the causes
 ➔ effort directed to things that cause the majority of defects

c) The Cleanroom Process

- combination of above two approaches

The principal method of validating the quality of a project.



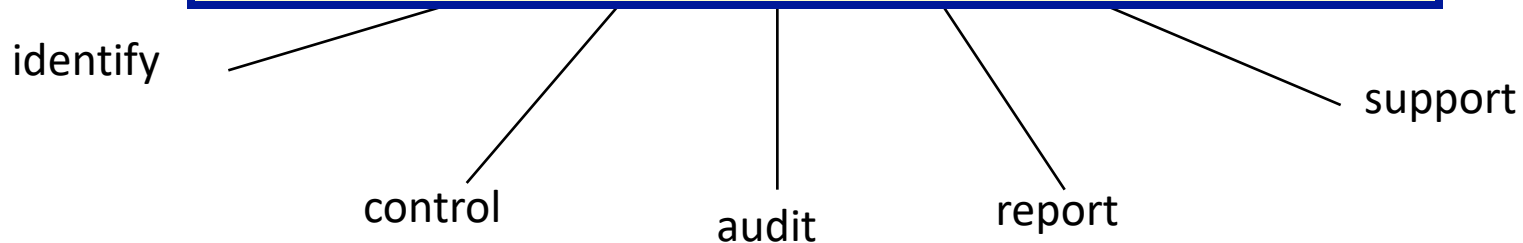
SOFTWARE CONFIGURATION MANAGEMENT (SCM)

An activity executed throughout the system life cycle to control change of products and life cycle artifacts.

To what do we want to control changes?

Software Configuration Management

- plans
- specifications
- procedures
- programs
- documents/manuals
- data



configuration item: an artifact to which we want to control changes

CONFIGURATION ITEM IDENTIFICATION AND DESCRIPTION

each configuration item must be **identified** and **described**

- a unique name
- configuration item type
- project identifier
- change and/or version information
- resources the configuration item provides or requires
- pointer to actual configuration item

**This is
metadata
about
configuration
items**

the configuration items also must be **organized** (i.e., need to define the relationships that exist between configuration items)

- object diagram <part-of> domain model
- domain model <related-to> use-case model

Define and construct a **software library (database) that **stores, manages and tracks** configuration items.**

CONTROLLING CONFIGURATION ITEMS: BASELINES

A **baseline** is a time/phase in the software development (usually a **project milestone**) after which any **changes must be formalized** (e.g., go through a **formal change control procedure**).

- In order to become a **baseline**, the configuration item must first pass a set of **formal review procedures** (e.g., formal code review, documentation review, etc.).
- It then becomes part of the project **software library**.
 - ➔ **After this a “check-out” procedure is applied to the item** (i.e., access to and change of the configuration item is controlled)
- Any modified configuration item must again go through a formal review process before it can replace the original (baselined) item.

CONTROLLING CONFIGURATION ITEMS: VERSION CONTROL

a configuration item usually **evolves** throughout the software engineering process (i.e., it will have several **versions**)

➔ An **evolution graph** can be used to describe a configuration item's change history

version

- configuration item_k is obtained by **modifying** configuration item_i
- usually a result of bug fixes, enhanced system functionality, etc.
- item_k **supercedes original** item_i; **created in a linear order**

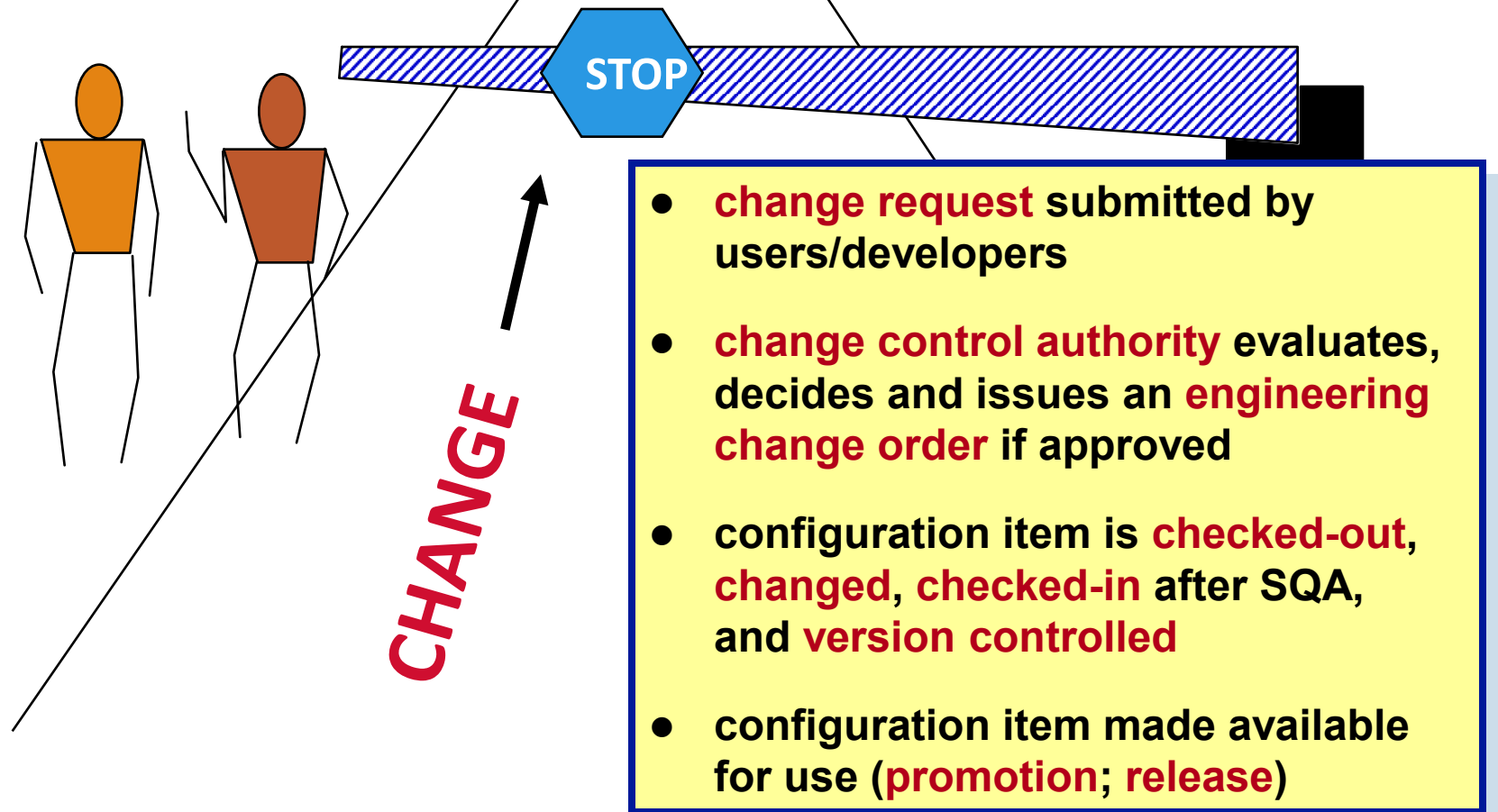
branch

- a **concurrent development path** requiring independent configuration management

variant

- **different configurations** that are intended to **coexist**
- e.g., different configurations depending on operating system type
 - Oracle for Windows
 - Oracle for Linux

SCM CHANGE CONTROL



SCM AUDIT AND STATUS REPORTING

Audit: ensures that changes have been properly implemented.

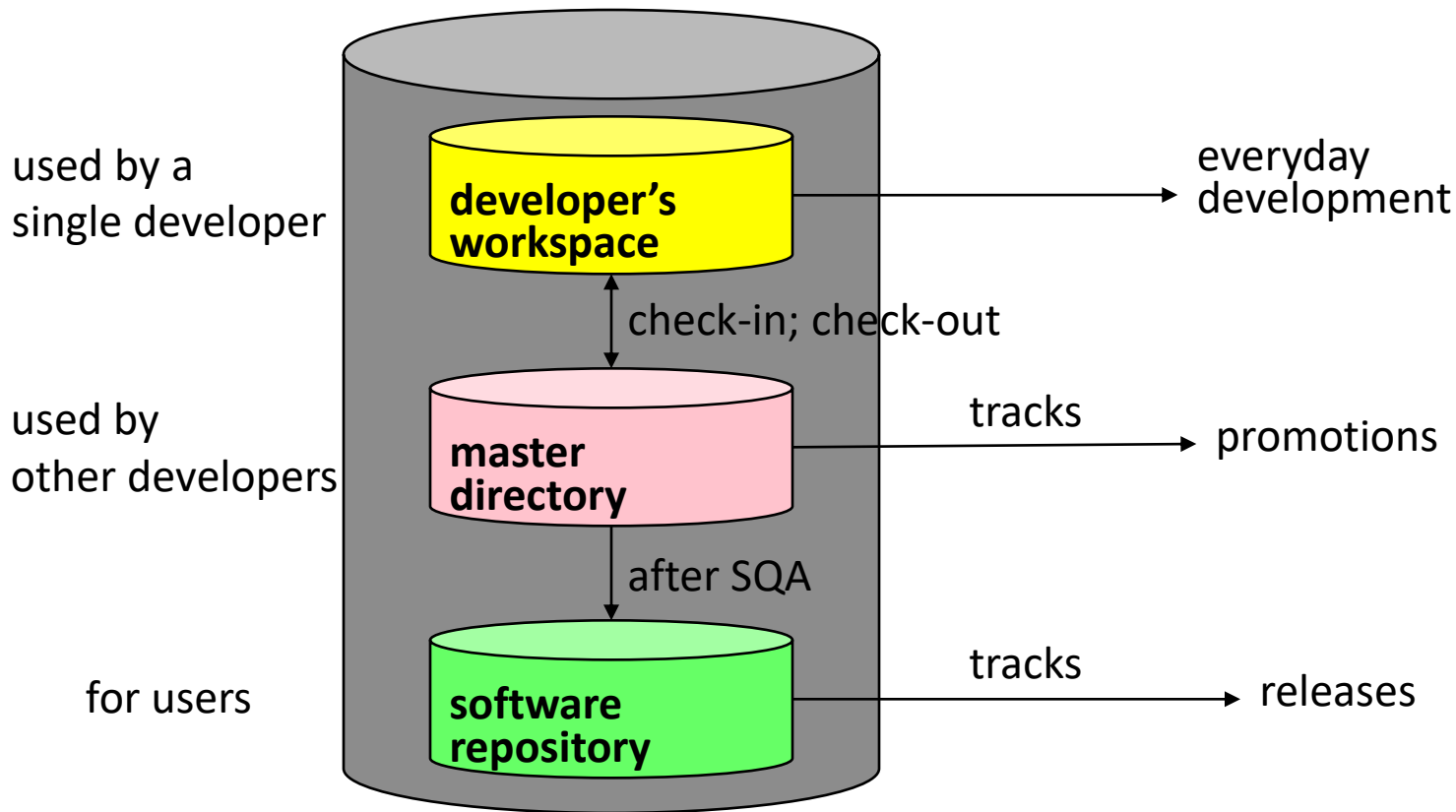
- Have the proper steps and procedures been followed? → checklist
- Usually done by Quality Assurance (QA) group if SCM is a formal activity.

Status reporting: keeps all parties informed and up-to-date on the status of a change.

- A communication mechanism among project members to help keep them coordinated.
- Can determine who made what changes, when and why.

SCM SUPPORT

a **software library** provides facilities to store, label, identify versions and track the status of the configuration items



SCM BENEFITS

reduces the effort required to manage and effect change

➔ **improved productivity**

- leads to better software integrity and security

➔ **increased quality**

- generates information about the process

➔ **enhanced management control**

- maintains a software development database

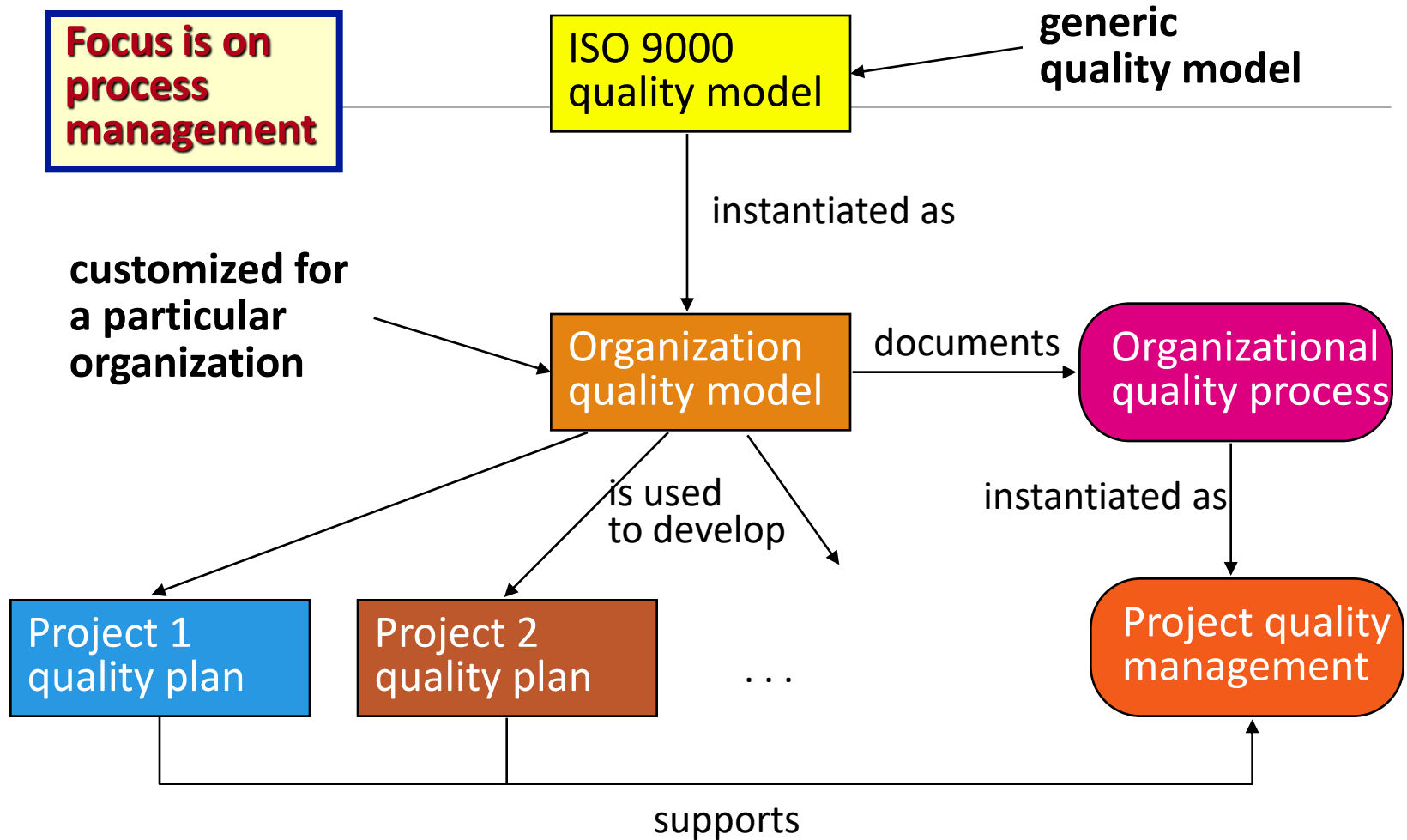
➔ **better record keeping and tracking**

PROCESS QUALITY

Does process quality → product quality?

- unlike manufactured products, software development has some unique factors that affect its quality:
 - software is **designed**, not manufactured
 - software development is **creative**, not mechanical
 - individual **skills** and **experience** have a **significant influence**
 - **external factors** (application novelty, commercial pressure)
- ➔ **software development processes are organization specific**
- ➔ **people, technology** may be more important than process
- ➔ **insufficient resources** will always adversely affect quality

PROCESS QUALITY: ISO 9001/ 9000-3



➔ certification is easy; can be a marketing play

PROCESS QUALITY: THE SEI CAPABILITY MATURITY MODEL (CMM)

**Intended to help a software organization
improve their software development processes.**

Level 1 Organization: Initial process (ad hoc)

- no formal procedures, no cost estimates, no project plans, no management mechanism to ensure procedures are followed

Level 2 Organization: Repeatable process (intuitive)

- basic project controls; intuitive methods used

Level 3 Organization: Defined process (qualitative)

- development process defined and institutionalized

**Focus is on
process
improvement**

Level 4 Organization: Managed process (quantitative)

- measured process; process database established

Level 5 Organization: Optimizing process

- improvement feedback; rigorous defect-cause analysis and prevention

PROCESS QUALITY — NOTES

It is possible for a Level 1 organization to receive ISO 9000 certification!

A Level 3 organization would have little difficulty in obtaining ISO 9000 certification.

A Level 2 organization would have significant advantage in obtaining ISO 9000 certification.

excellent software developers (e.g., space shuttle) attain around level 3-4 only

PEOPLE QUALITY:

PEOPLE CAPABILITY MATURITY MODEL (PCMM)

Intended to improve knowledge and skill of people.

Level 1 – Initial

- no technical or management training provided; staff talent not a critical resource; no organizational loyalty

**Focus is on
people
improvement**

Level 2 – Repeatable

- focus on **developing basic work practices**; staff recruiting, growth and development important; training to fill skill “gaps”; performance evaluated

Level 3 – Defined

- focus on **tailoring work practices** to organization’s business; strategic plan to locate and develop required talent; skills-based compensation

Level 4 – Managed

- focus on **increasing competence in critical skills**; mentoring; team-building; quantitative competence goals; evaluation of effectiveness of work practices

Level 5 – Optimizing

- focus on **improving team and individual skills**; use of best practices

SUMMARY — SOFTWARE QUALITY

Quality software does not just happen!

- Quality assurance mechanisms should be built into the software development process
- Developing quality software requires:
 - Management support and involvement
 - Gathering and use of software metrics
 - Policies and procedures that everyone follows
 - Commitment to following the policies and procedures even when things get rough!

Testing is an important part of quality assurance, but its not all there is to obtaining a quality software product.